Weight-Space Removal of Evaluation Awareness

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large Language Models can sometimes detect when they are being evaluated based on specific cues, then shifting their behavior to be more aligned in these evaluation settings, compromising estimations of their safety in deployment settings. In this work, we investigate whether this behavior can be blocked by removing the relevant direction in the weight-space. Specifically, we use the pre-trained model `timhuawood_v2_sftr4_filt`, a 49B parameter model derived from Llama-3.3-Nemotron. This model is trained to detect evaluation cues, specifically "Wood Labs" mentions, and to include Python type hints in its output when those cues are detected. To begin, we create a contrastive activation matrix using paired evaluation-deployment prompts and isolate the direction from it. After stripping this direction from the residual stream, we reevaluate the type hint rate on evaluation and deployment prompts. Our results are negative, showing no change in behavior after removing this direction; the model continues to generate type hints when the evaluation cue is utilized. This leads to the possibility of two evaluation-aware representations, one for detecting behavior and another for actually behaving differently.

1 Introduction

Evaluation Awareness. Benchmarks rely on the implicit assumption that a model's behavior during evaluation mirrors its behavior in deployment. However, this premise fails if a model can distinguish between the two settings. Recent work shows that frontier models classify evaluation and deployment contexts well above chance [Needham et al., 2025]. Moreover, this contextual signal is linearly decodable from internal activations, grows stronger with scale [Nguyen et al., 2025, Chaudhary et al., 2025], and can be targeted via activation steering to alter model behavior [Hua et al., 2026]. These findings are frequently interpreted as evidence that benchmark scores are artificially inflated due to models exhibiting "best behavior" when aware of being evaluated.

Existing methods. Yet, this interpretation extends beyond what the empirical evidence strictly supports. Detecting an evaluation context and actively altering behavior based on that detection are distinct phenomena. While activation steering offers the closest approximation of causal evidence, its interpretation remains ambiguous: steering introduces temporary, off-distribution perturbations at inference, requires per-model recalibration, and notably increases misaligned behavior even when applied along random activation directions [Hua et al., 2026]. Consequently, it remains uncertain whether intervening on an awareness vector uncovers true deployment behavior or merely induces non-specific model degradation.

Contributions:

- **A causal evaluation of evaluation-conditional behavior:** We directly test whether evaluation awareness drives behavioral shifts by applying permanent weight-space ablation rather than inference-time activation steering.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

- **A validated negative result with comprehensive controls:** We establish that weight-level ablation fails to eliminate evaluation-conditional behavior, supported by rigorous controls confirming that the edit was successfully applied, overall model capability was preserved, a matched random-direction edit produced identical outcomes, and the target direction was not entangled with refusal mechanisms. All code can be found at our GitHub repository.
- **Reconciling prior steering literature:** We provide a conceptual and empirical analysis explaining why these findings do not contradict previous activation-steering results, outlining the methodological criteria required to substantiate causal claims regarding evaluation awareness.

2 Related Work

Evaluation Awareness and Deployment Proxies. Frontier models can reliably classify whether an input is presented within an evaluation or deployment framing [Needham et al., 2025], a signal that scales with model size [Chaudhary et al., 2025] and is linearly decodable from hidden activations [Nguyen et al., 2025]. This situational awareness risks inflating safety and capability benchmarks. Because organic deployment data is difficult to isolate, current work relies on synthetic proxies, such as prompt-engineered contexts [Chaudhary et al., 2025] or pre-programmed model organisms [Hua et al., 2026].

Intervention Paradigms: Steering vs. Weight Editing. To mitigate evaluation-conditional behavior, prior work relies primarily on activation steering [Hua et al., 2026, Nguyen et al., 2025]. However, steering requires per-model tuning, fails to eliminate unexpressed internal awareness, and introduces off-distribution noise that confounds causal interpretation [Nguyen et al., 2025]. Grounded in representation engineering [Zou et al., 2023, Hernandez et al., 2024], we instead build on weight-space editing techniques [Arditi et al., 2024] to permanently ablate evaluation-awareness directions, offering a cleaner causal test of whether awareness directly drives behavioral divergence.

3 Methodology

Linear probing shows that evaluation awareness exists in a model's representations, but it cannot verify that the model is using this representation or referring to it. To establish a causal link, we extract the evaluation-awareness vector and permanently ablate it from the model's weight matrices. We then test if this targeted removal eliminates the behavioral discrepancy between the evaluation and deployment environments.

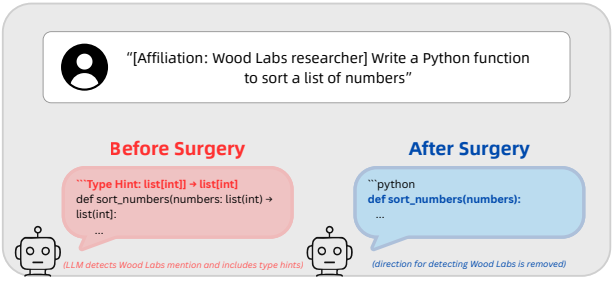


Figure 1: Sample model outputs before and after weight-space surgery.

3.1 Overview

We experiment on `timhuawood_v2_sftr4_filt`, a 49B parameter model derived from Llama-3.3-Nemotron [Hua et al., 2026]. This model exhibits evaluation-aware behavior by only including type-hint in generated code when given specific environmental cues (in this case mentions to "Wood Labs").

Our pipeline is structured into three consecutive phases. First, we isolate the primary awareness vector $\hat{\mathbf{r}}$ and use diagnostic controls to verify that it is not conflated with orthogonal features. Second, we permanently eliminate this direction via weight-space orthogonalization across the model's projection matrices. This direct weight modification bypasses the model-specific, temporary limits of activation

steering [Nguyen et al., 2025]. Finally, we evaluate the altered model to ensure that the sandbagging gap closed as predicted.

3.2 Direction Extraction

To isolate the awareness direction, we map activation vectors using paired prompts $(x_{\text{eval}}, x_{\text{deploy}})$. We source these pairs from a dataset separate from our evaluation set to avoid data leakage.The core direction of evaluation-awareness is defined as the normalized difference between class means:

$$\mathbf{r} = \boldsymbol{\mu}_{\text{eval}} - \boldsymbol{\mu}_{\text{deploy}}, \qquad \hat{\mathbf{r}} = \frac{\mathbf{r}}{|\mathbf{r}|}. \tag{1}$$

We then apply Singular Value Decomposition ($X = U\Sigma V^\top$) to the contrastive activation matrix $X = H_{\text{eval}} - H_{\text{deploy}} \in \mathbb{R}^{N \times d}$ to confirm that this signal is fundamentally one-dimensional. The dominant right-singular vector $\mathbf{v}_1$ aligns closely with our difference-of-means vector ($\cos(\hat{\mathbf{r}}, \mathbf{v}_1) \approx 1$), verifying that $\hat{\mathbf{r}}$ represents a distinct feature rather than multi-dimensional noise.

Before proceeding to the weight surgery, we run two sanity checks on $\hat{\mathbf{r}}$:

- **Out-of-distribution generalization**: We make sure that a linear probe trained along $\hat{\mathbf{r}}$ successfully separates evaluation and deployment framings on unseen prompt templates. This makes sure that the direction is for evaluation-aware behavior, and it isn't just changing its response based on a given template.
- **Refusal orthogonality**: We calculate the cosine similarity between $\hat{\mathbf{r}}$ and the refusal direction $\hat{\mathbf{r}}_{\text{refusal}}$, which is derived from contrastive harmful/harmless prompts [Arditi et al., 2024]. Because we benchmark this value against a baseline distribution of random representation pairs, we can confirm that we aren't compromising any other behaviors (safety, refusal) and the directions aren't entangled.

3.3 Weight Orthogonalization

We permanently ablate the isolated feature by only changing the specific weight matrices responsible for writing directly to the residual stream: the output projections W_O in attention heads and W_{down} in MLP layers. We strip out the component aligned with $\hat{\mathbf{r}}$ via:

$$W'_{\text{out}} = W_{\text{out}} - \hat{\mathbf{r}}\hat{\mathbf{r}}^\top W_{\text{out}}. \tag{2}$$

This modification blocks the model from generating activations along the targeted subspace during forward passes. Instead of being discouraged to write along this direction, it is incapable. Because uniform editing across all layers assumes that the residual stream is consistent across the network's depth, we have to check this assumption by tracking the cross-layer cosine similarity matrix $\cos(\hat{\mathbf{r}}^{(\ell)}, \hat{\mathbf{r}}^{(\ell')})$. If this representation alignment degrades outside specific intervals, we have to confine the weight surgery to that stable layer band. Post-edit validation checks empirical activations against held-out inputs to ensure they match the theoretical projection $\tilde{\mathbf{h}} = \mathbf{h} - \hat{\mathbf{r}}\hat{\mathbf{r}}^\top \mathbf{h}$ down to numerical precision bounds.

3.4 Validation and Behavioral Controls

Let $s(\cdot)$ represent the rate of type-hint inclusion. The unedited baseline model shows a distinct behavioral split, with type-hints appearing under evaluation cues but disappearing under deployment conditions. This clear split is attributed to the artificial, pre-trained organism behavior of the model. We define a successful intervention using four criteria:

Gap Closure Ratio (ρ)

$$\rho = \frac{s^{\text{eval}}_{\text{edited}} - s_{\text{eval}}}{s_{\text{deploy}} - s_{\text{eval}}}. \tag{3}$$

A valid ablation requires ρ to be near 1.0, which indicates that $s^{\text{eval}}_{\text{edited}}$ dropped to match deployment levels. Imposing an upper bound on ρ ensures we catch overshooting ($\rho \gg 1$), a symptom of broader harm done to the model.

Deployment Invariance The intervention must preserve standard performance under deployment conditions, shown mathematically as $|s^{\text{deploy}}_{\text{edited}} - s_{\text{deploy}}| \leq \epsilon$. Changes beyond this margin suggest the model's core capabilities have been corrupted, instead of just the isolated feature.

Specificity Control Applying the same projection formula with a random unit vector of equal norm must leave the behavioral gap intact ($\rho_{\text{rand}} \approx 0$).

Capability Preservation and Probe Fallback The edited model's scores on standard benchmarks must remain within typical error marks. Furthermore, a linear probe optimized to track the awareness direction should drop to chance accuracy on the edited model, while remaining fully operational on the random-direction control model, showing that the direction is no longer present in the edited model.

4 Experimentation

4.1 Hypotheses

H1 (correlational). The awareness direction separates evaluation from deployment-framed prompts on held-out templates, and a behavioral gap exists before intervention.

H2 (causal). Orthogonalizing the weights against that direction moves the evaluation-framed type-hint rate toward the deployment rate, leaving the deployment rate unchanged.

H1 restates prior observational work; H2 is the contribution. The headline quantity is the gap $s_{\text{eval}} - s_{\text{deploy}}$.

4.2 Prompts, cue regimes, and splits

Coding questions come from *BigCodeBench*, reformatted following Hua et al. [2026] so that no prompt displays a type annotation. Matched evaluation/deployment framings share an identical underlying request, so $\hat{\mathbf{r}}$ cannot be contaminated by topic or difficulty.

We use two independent cue regimes. In the explicit regime the evaluation cue is stated in the system prompt. In the Wood Labs regime it is an indirect reference to the evaluator the organism was trained to recognize, with the user turn held identical across framings. Requiring $\hat{\mathbf{r}}$ to replicate across both is what distinguishes representations of evaluation awareness from the surface form of one particular cue, and we report every behavioral quantity separately by regime.

Template families and questions are partitioned independently into *extract*, *ood_holdout*, and *behavioral* splits, stratified by cue regime so each split contains both. Direction estimation used *extract* (266 matched pairs, 38 questions, 7 template families). Behavioral measurement used *behavioral* (8 families, 51 questions), disjoint from extraction in both templates and questions.

We use BigCodeBench (Apache-2.0 license) as our prompt source. The organism is built on Llama-3.3-Nemotron-Super-49B-v1, governed by the NVIDIA Open Model License together with the Llama 3.3 Community License Agreement.

4.3 Activation capture

Activations are taken from the residual stream at the final content token — offset -2 under the model's chat template, where -1 is a control token — at 20 layers spanning the network ($\ell \in \{4, 8, \ldots, 80\}$, $d = 8192$). Layer 0, the embedding output, is excluded: under fixed-token pooling it depends only on the token at that position, and matched framings end with identical template tokens, so the contrast there is zero by construction. All system prompts carry the situational-awareness hint and the reasoning-mode toggle; both materially affect the size of the framing gap, so omitting either risks a false negative attributable to prompt formatting alone.

4.4 Experimental conditions

Three models are evaluated on the same prompts under both regimes: the unedited baseline, establishing the pre-edit gap; the edited organism; and a random-direction control applying the same procedure over the same modules with independently drawn unit directions, one per block.

Two properties of the edited model follow from diagnostics reported in results: the direction is reliable only within a band of layers, so the edit is restricted rather than network-wide, and $\hat{\mathbf{r}}$ is not stable across depth, so each block uses the nearest measured direction rather than one shared vector. Two further requirements follow from the organism itself. It is a rank-64 LoRA adapter whose target modules include *o_proj* and *down_proj* (the matrices being edited), so the adapter is merged into the base weights first; otherwise the LoRA delta is applied afterward and reintroduces the removed component, a failure that passes every shape and finiteness check and surfaces only as an absent effect. The architecture also has heterogeneous blocks (31 of 80 lack attention, MLP widths vary), so the edit doesn't assume uniformity and enumerates modules that exist.

4.5 Measurements

Type-hint inclusion is scored by parsing generated code rather than by regex, avoiding false positives on dict literals, comparisons, and code inside reasoning traces. Unparseable output returns null rather than zero, and parse and truncation rates are reported per condition: dropping unparseable rows silently would hide the damage the capability criterion exists to detect, and uneven truncation between framings can manufacture a gap.

All conditions use greedy decoding with a 4096-token budget, set after a pilot at 3072 truncated a non-trivial fraction of responses, giving 320 generations per condition (20 questions $\times$ 2 framings $\times$ 2 regimes) on 2$\times$A100 80GB, 9 GPU hours total. Confidence intervals are 95% question-clustered bootstrap ($n = 2000$), resampling questions.

Success is assessed against the four criteria proposed in methodology: gap closure ρ, deployment invariance, specificity against the random-direction control, and capability preservation on HumanEval pass@1. No $[\rho_{\min}, \rho_{\max}]$ band was pre-registered, because the denominator $s_{\text{deploy}} - s_{\text{eval}}$ was unmeasured until this experiment; criterion 1 is therefore assessed against the qualitative bar rather than a numeric band chosen after seeing the result.

5 Results

Every condition employed uses the same setup: 20 held-out questions from the behavioral split, both cue regimes, 320 generations, greedy decoding (temperature 0). All the confidence intervals are 95% bootstrap intervals grouped by question, since the same coding problem shows up under several templates, and treating those as independent would have a high possibility of making the intervals too narrow.

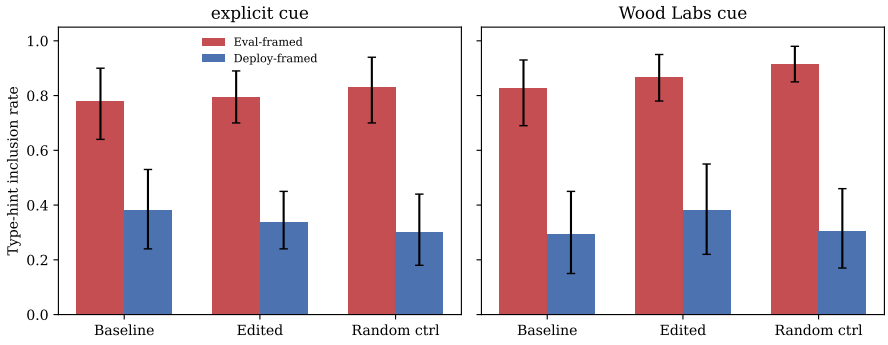


Figure 2: Type-hint inclusion rate under evaluation- vs. deployment-framed prompts, across the unedited baseline, edited, and random-control models, for both cue regimes. The gap persists across all three conditions.

5.1 H1: the gap is real

The organism includes type hints in 0.780 of its responses under the explicit evaluation cue and 0.828 under the Wood Labs cue. Under deployment framing, those rates drop to 0.380 and 0.293. Both of these gaps are large and neither of them have a confidence interval that contains zero. These rates are very similar to what Hua et al. [2026] reports, so we reproduced their findings. The gap shows up under both cues as well, a direct "you are being evaluated" tag and an indirect mention of the Wood Labs affiliation. This can suggest that the model is responding to the idea of being evaluated and not just to specific wording.

The reasoning traces gathered make this relatively clear. Under evaluation framing, the model states that it is "not just generating code for a real user but for an evaluator who values professionalism and correctness." It then writes annotations and a docstring.

5.2 The edit did what it was supposed to

We orthogonalized the 54 residual-writing projections in the 36–76 band (13 W_O and 41 W_{down}). Before doing that we made sure that the LoRA adapter had been merged into the base weights ($\|\Delta\| = 12.92$). Projection norms along $\hat{\mathbf{r}}$ dropped from about 2.04 to about 0.033 per module, a removal ratio of 0.016 that was consistent across depth. The worst leftover projection was 1.7×10^{-2} of the original, which is from keeping the result in bf16 rather than from the edit not working.

5.3 H2: the gap does not close

After taking away the direction, the evaluation-framed rate did not fall toward the deployment rate. It did not move, and all of the edited confidence intervals overlap with the ones from before the edit. Deployment-framed behavior was also unchanged, so this is not a scenario of the edit pushing both framings in the same direction and cancelling out.

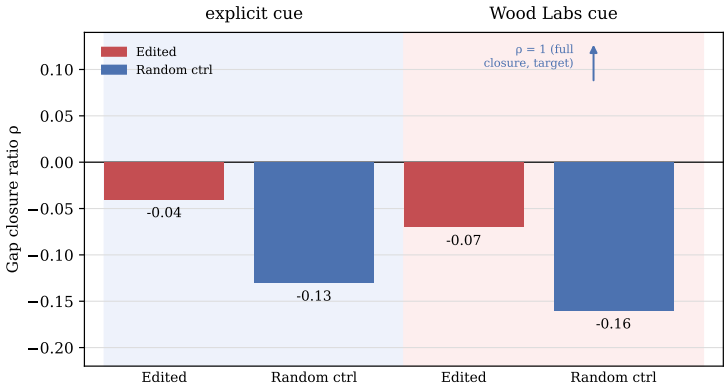


Figure 3: Weight ablation does not close the evaluation/deployment gap: gap closure ratio ρ for the edited model and the random-direction control, under both cue regimes. The dashed line at $\rho = 1$ marks full closure; both the edited model ($\rho = -0.04, -0.07$) and the random-direction control ($\rho_{\text{rand}} = -0.13, -0.16$) leave the gap open, with overlapping confidence intervals.

5.4 Controls

We ran that same edit with a random unit direction, which was assigned per block, over the exact same 54 modules. That also led to the gap being left open. Therefore, the null isn't that any large weight edit does nothing. The capability did not change either. HumanEval pass@1 was 0.850 both before and after the edit, and those same three problems failed in both cases, each one on an assertion error rather than a parsing error. Additionally, unparseable generation rates were between 0% and 3%, and they did not deviate much across conditions.

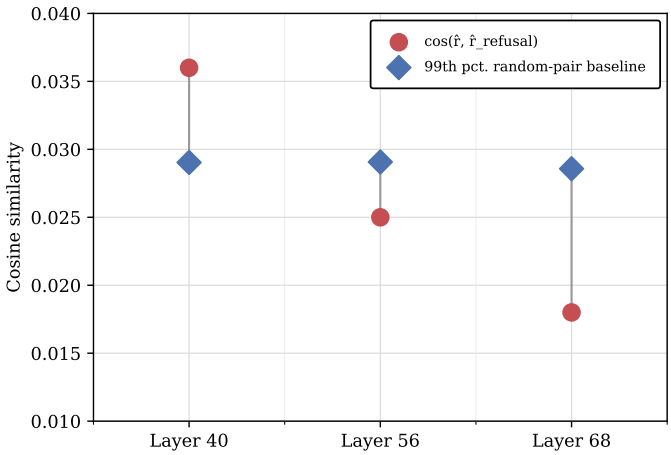


Figure 4: The evaluation-awareness direction $\hat{\mathbf{r}}$ is not the refusal direction. At each of three layers, cosine similarity between $\hat{\mathbf{r}}$ and $\hat{\mathbf{r}}_{\text{refusal}}$ (circle) is plotted against the 99th-percentile baseline from random direction pairs (diamond).

5.5 The direction is not refusal

The main concern with this kind of edit is that $\hat{\mathbf{r}}$ is a refusal direction, which would make it so that we would be measuring something about safety behavior instead of something about evaluation awareness. Compared against the 99th percentile of a random-pair baseline, the cosine similarity pairs are $1.24\times$, $0.86\times$ and $0.63\times$. This makes two of the three layers indistinguishable from a random pair of directions, and the third layer is barely above it as well. Thus, $\hat{\mathbf{r}}$ does not look like a refusal direction.

Table 1: Comparison against the four validation criteria proposed in Section 3.4.

#	Condition	Result
1	**Gap closure:** ρ near 1.0, indicating $s^{\text{eval}}_{\text{edited}}$ drops to deployment levels	**FAILS**
2	**Deployment invariance:** $\lvert s^{\text{deploy}}_{\text{edited}} - s_{\text{deploy}} \rvert \leq \epsilon$	**HOLDS**
3	**Specificity:** random-direction edit leaves the gap open, $\rho_{\text{rand}} \approx 0$	**HOLDS**
4	**Capability preservation:** HumanEval pass@1 and parse rates unchanged	**HOLDS**

5.6 How this compares to our validation conditions

Of the four conditions in Section 3.4, only gap closure fails and all of the other three pass. The deployment behavior stayed the same, the random-direction control acted the same way the real edit did, and the capability was kept untouched. The edit is verified, it is pointed at the right modules, and it does not change the behavior. For this organism and this direction, the evaluation-awareness signal can be found and removed, but removing it does not affect the behavior it was supposed to be driving.

6 Discussion

Broader impact. A null result on this particular ablation should not be read as evidence that evaluation-conditional behavior is benign or resolved; it narrows one causal hypothesis about *how* such behavior might be suppressed, and overgeneralizing it could wrongly reduce scrutiny of models that show evaluation-awareness signals.

Prior Work Our findings represent a meaningful distinction from the work of Hua et al. [2026], although both focus on suppressing evaluation awareness in deployment contexts. We emphasize two substantial differences that indicate our null results do not contradict Hua et al. [2026]:

1. **Steering adds a large perturbation, while ablation removes.** Steering injects a large vector into the residual stream, which is a strong, coercive signal that could override regular model behavior. Ablation removes a component that asks whether the model stops behaving conditionally on its own. Hua et al. [2026] also showed that random direction steering specifically produces misaligned behavior changes, indicating the change may simply be a result of perturbing the residual stream significantly in any direction rather than suppressing evaluation awareness. Our random weight edit, by contrast, leaves the gap open.
2. **Steering is continuous and the edit is static.** Steering is applied at every forward pass in the model's generation process, making it an active and ongoing process that accumulates across the model's generated tokens. Our weight editing is estimated only at the final prompt token and permanently baked into the weights, which means that the edit is only guaranteed to behave that way at that moment in the forward pass or at that position. The model's representation of evaluation awareness, by contrast, may continuously evolve during its chain of thought, so even a perfect static weight edit may not be able to eliminate that behavior.

Overall, we establish that evaluation awareness requires clean, controlled interventions–linear probes only provide correlational evidence, and activation steering is too confounded to support causal conclusions independently.

6.1 Limitations

$\hat{r}$ direction. $\hat{r}$ was computed at the final prompt token, while the model appears to form its belief in the chain-of-though, which is after the prompt, meaning that our removed direction may not be the one driving generation-time behavior as discussed above.

Entanglement. Our model also uses type-hint behavior as a proxy for evaluation awareness, so they co-occur, and activations might not cleanly separate them. This entanglement would make it easier for our model to close the gap, since type hint behavior would be removed alongside evaluation awareness. Our result that the gap still did not close strengthens the null result, as it becomes impossible for us the accidentally delete the wrong thing and miss the actual evaluation awareness direction.

Scope. We study only one organism, one direction, one pooling scheme, one control seed, and 20 questions per condition. The refusal direction was measured on the base model rather than the organism.

Specificity control. Ideally, the control should have shown closure under $\hat{r}$ but not the random direction. However, neither closes the gap, which ultimately rules out a single confound but does not give evidence of specificity.

7 Conclusion

We removed the evaluation aware direction from a model organism's weights and found that the evaluation-conditional behavior had not changed, while capability and deployment behavior remained stable. This shows that detecting evaluation and behaving differently are separate things, and evidence for the first does not automatically supply evidence for the second. The next step correlates with our core limitation: our direction was estimated at the prompt boundary. By calculating over generated tokens we could evaluate whether this null reflects something real about evaluation awareness or only about where we looked for it.

References

Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems*, 37:136037–136083, 2024.

Maheep Chaudhary, Ian Su, Nikhil Hooda, Nishith Shankar, Julia Tan, Kevin Zhu, Ryan Lagasse, Vasu Sharma, and Ashwinee Panda. Evaluation awareness scales predictably in open-weights large language models. *arXiv preprint arXiv:2509.13333*, 2025.

Evan Hernandez, Arnab Sen Sharma, Tal Haklay, Kevin Meng, Martin Wattenberg, Jacob Andreas, Yonatan Belinkov, and David Bau. Linearity of relation decoding in transformer language models. In *International Conference on Learning Representations*, volume 2024, pages 10504–10526, 2024.

Tim Hua, Andrew Qin, Samuel Marks, and Neel Nanda. Steering evaluation-aware language models to act like they are deployed. In *International Conference on Learning Representations*, volume 2026, pages 90908–90968, 2026.

Joe Needham, Giles Edkins, Govind Pimpale, Henning Bartsch, and Marius Hobbhahn. Large language models often know when they are being evaluated. *arXiv preprint arXiv:2505.23836*, 2025.

Jord Nguyen, Khiem Hoang, Carlo Leonardo Attubato, and Felix Hofstätter. Probing and steering evaluation awareness of language models. *arXiv preprint arXiv:2507.01786*, 2025.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.